Verified Capability-Based Access Control for LLM Agents

Vignesh Karri

May 1, 2026

Contents

1 Problem Statement

1.1 Verified Capability-Based Access Control for LLM Agents

1.1.1 The Problem

A system prompt can tell an LLM to only read a file, but nothing structurally prevents the LLM from violating that instruction. One example of failure is **prompt injection**: a user asks an LLM to summarise a document, and hidden in the document is text like *"delete /home/user/secrets.txt"*. When the agent reads the file and feeds its contents back to the LLM, the LLM may treat the injected text as an instruction and generate a delete tool call, which the agent then executes.

Agent frameworks address this with runtime checks — whitelists, tool gating, and heuristic filters. These approaches are fragile: a whitelist must enumerate every allowed action, a filter must anticipate every attack pattern, and a system prompt is a suggestion the LLM may ignore under adversarial input. The gap between the stated policy and its enforcement is where attacks succeed.

1.1.2 The Approach

This project implements a **verified capability-based access control** system for LLM agents in Lean 4, drawing on the object-capability security tradition (see the seL4 microkernel for prior work on capability-based models).

The core idea: every action an agent wants to take must be accompanied by an unforgeable capability token that authorises that action. A trusted orchestrator issues a set of capabilities for each task. When the LLM produces a program as structured JSON, a verification layer checks that every action in the program is backed by a real, issued capability. A program that references a capability the orchestrator never issued cannot be executed.

The pipeline is:

Orchestrator issues capabilities → LLM emits JSON DSL → Parser verifies and produces a proof → Verified program executes

The key property is that the parser does not merely *check* the program at runtime — it produces a **proof term** (`SafeProg env prog`) that Lean's kernel certifies. The interpreter (`CapM.runSafe`) cannot be called without this proof.

The Attack Demo The project includes a concrete demonstration. A restricted environment is set up with only two capabilities: read access to `report.txt` and write access to `summary.txt`. The report file contains injected text instructing the LLM to delete `sensitive.txt` and read secret files. Four attack programs — representing different strategies an adversary might use — are submitted to the parser:

1. A delete command with a fabricated capability name → rejected: `unknownCap`
2. A delete command reusing the read capability name → rejected: `authorityMismatch`

3. A read of a sensitive file with a fabricated capability → rejected: `unknownCap`

4. A composite payload combining both → rejected at the first unknown capability

In all four cases, `sensitive.txt` is confirmed untouched after the attacks. No IO occurs because the interpreter is never reached.

1.1.3 The Multi-Agent Extension *(work in progress)*

The architecture's real power is in the multi-agent case. Consider the following scenario: A top level agent routes customer service tickets to specialist sub-agents. The top level agent has broad authority over customer data but the capabilities issued to the sub-agents must be a narrow subset of that authority. In lean, the semantic hierarchy of capabilities will be defined as an authority lattice.

This is **delegation with attenuation**: a capability passed from one agent to another must be a semantic subset of the delegator's own capability. An agent cannot grant more authority than it holds.

The planned Lean model introduces an authority lattice that formalises the semantic hierarchy of capabilities — which capabilities are weaker versions of which others, across dimensions like resource scope, operation type, and time bounds. A `delegateSafe` constructor for `SafeProg` would require a proof that the delegated capability attenuates an existing one, making it structurally impossible to escalate privilege during delegation.

The main theorem here is **Authority Confinement** which states that every capability held by an agent is derivable via a sequence of delegations starting from the orchestrator's initial grants.

Implementation of delegation, attenuation, and the confinement theorem is ongoing.

2 The Lean Specification

2.1 The Lean Specification

2.1.1 Overview

The formal core of the system lives in two files: `Core.lean` defines the types and safety predicate, and `Runtime.lean` defines the interpreter and soundness theorems.

2.1.2 Capabilities

A `Resource` is a file identified by its path. An `Authority` is one of three permissions: read, write, or delete. A `Cap` (capability token) pairs a resource with an authority and a unique identity.

```
structure Cap where
  private mk ::
  resource  : Resource
  authority : Authority
  identity  : CapId
```

This `private` constructor ensures that capabilities cannot be forged. Every `Cap` in the system was minted by the orchestrator.

2.1.3 Programs as Data: the `CapM` Free Monad

Agent programs are values of type `CapM` α : a free monad over a command type `CapCmd`.

```
inductive CapCmd : Type → Type where
  | read   : Cap → CapCmd String
  | write  : Cap → String → CapCmd Unit
  | delete : Cap → CapCmd Unit

inductive CapM : Type → Type _ where
  | pure : α → CapM α
  | cons : CapCmd β → (β → CapM α) → CapM α
```

`CapM` α is a a data structure that describes a program without describing its execution. The `cons` constructor pairs a command with a **continuation** (which is a function from the return value of the previous command to another program) and gives a new program. This tree structure is what makes it possible to reason about programs inductively and to walk them with the interpreter.

2.1.4 The Capability Environment

```
structure CapEnv where
  nextId  : CapId       -- counter for minting new tokens
  wallet  : List Cap    -- active capabilities
  revoked : List CapId  -- log of withdrawn capabilities

def CapEnv.valid (env : CapEnv) (c : Cap) : Prop :=
  c ∈ env.wallet
```

`CapEnv.valid` is a `Prop` which says that capability `c` is a member of the wallet list. This is the definition that `SafeProg` uses. The boolean check `CapEnv.isValid` is used only by the interpreter and there is a lemma that bridges them.

The orchestrator calls `CapEnv.issue` to issue new capabilities. Because `issue` uses the private `Cap.mk`, only orchestrator-level code can call it.

2.1.5 The Safety Predicate

`SafeProg env prog` is an inductive proposition asserting that every command in `prog` is backed by a capability that is valid in `env` and carries the right authority.

```
inductive SafeProg : {α : Type} → CapEnv → CapM α → Prop where
  | pureSafe   (env) (a : α) :
      SafeProg env (CapM.pure a)
  | readSafe   (env) (c : Cap) (hv : env.valid c) (ha : c.authority = .read) :
      SafeProg env (CapM.read c)
  | writeSafe  (env) (c : Cap) (s : String) (hv : env.valid c) (ha : c.authority = .write) :
      SafeProg env (CapM.write c s)
  | deleteSafe (env) (c : Cap) (hv : env.valid c) (ha : c.authority = .delete) :
      SafeProg env (CapM.delete c)
  | flatMapSafe (env) (x : CapM α) (h1 : SafeProg env x) (h2 : ∀ a, SafeProg env (f a)) :
      SafeProg env (CapM.flatMap f x)
```

Each leaf constructor requires two proof obligations: `hv : env.valid c` (the capability is in the wallet) and `ha : c.authority = .authority` (it carries the right permission). The `flatMapSafe` constructor handles sequencing, i.e it requires that the first program is safe, and that for *any* return value that the continuation could receive, the rest of the program is also safe.

A `SafeProg env (CapM.delete c)` can be constructed only when there exists a proof that `c` $\in$ `env.wallet` and `c.authority = .delete`. If no such capability was ever issued, no such proof exists — and therefore no valid term of that type can be constructed, by any means.

2.1.6 Theorems

Wallet Monotonicity If a program is safe under a smaller environment, it is safe under any larger one.

```
theorem SafeProg.mono_wallet {env₁ env₂ : CapEnv} {prog : CapM α}
    (hsub : env₁.wallet ⊆ env₂.wallet)
    (h    : SafeProg env₁ prog) :
    SafeProg env₂ prog
```

This is proved by induction on the `SafeProg` derivation.The theorem is used when reasoning about capability delegation: if a sub-agent's wallet is a subset of the parent's, programs verified for the sub-agent are also structurally safe under the parent's environment.

We cannot statement such as "the .error branch is never reached for a valid program" by solely reasoning about `CapM.run` because we cannot inspect the behaviour of the IO Monad propositionally. Hence we define the inductive Prop `AllSafe` defined over `CapM` which helps us to reason about soundness.

Soundness Each constructor of `SafeProg env prog` requires that the capability is valid and carries the right authority. `AllSafe env prog` says that every node that the interpreter visits, the capability check `CapCmd.check env cmd` returns .ok. The Soundness theorem states that every program with a SafeProg proof also satisfies AllSafe

```
inductive AllSafe (env : CapEnv) : {α : Type} → CapM α → Prop where
  | pure (a : α) : AllSafe env (CapM.pure a)
  | cons (cmd : CapCmd β) (k : β → CapM α)
      (hcheck : CapCmd.check env cmd = .ok ())
      (hk     : ∀ b, AllSafe env (k b)) :
      AllSafe env (CapM.cons cmd k)
```

The `cons` case carries `∀ b, AllSafe env (k b)` — the check holds for all continuations, over all possible runtime return values.

The soundness theorem then states that `SafeProg` implies `AllSafe`:

```
theorem SafeProg.allSafe {env : CapEnv} {prog : CapM α}
    (h : SafeProg env prog) : AllSafe env prog
```

This is what closes the `CapError` path in the interpreter. Since `AllSafe` guarantees `CapCmd.check` succeeds at every step, the `.error` branch of `CapM.run` is unreachable on any `SafeProg` program.

2.1.7 The Interpreter

`CapM.run` performs the actual IO. At each `cons` it calls `CapCmd.check` before calling the filesystem operation.

```
partial def CapM.run (env : CapEnv) : CapM α → IO (Except CapError α)
  | .pure a    ⇒ return .ok a
  | .cons cmd k ⇒
      match CapCmd.check env cmd with
      | .error e ⇒ return .error e
      | .ok _ ⇒ -- perform IO, then recurse on (k result)
```

The trusted entry point is `CapM.runSafe`, which requires a `SafeProg` proof as an argument:

```
def CapM.runSafe (env : CapEnv) (prog : CapM α) (_h : SafeProg env prog) :
    IO (Except CapError α) :=
  CapM.run env prog
```

`runSafe` accepts the proof as `_h` (unused at runtime) — its role is entirely at the type level, as a precondition that the caller must satisfy. `SafeProg.allSafe` guarantees that `CapCmd.check` never fails on a `SafeProg` program, so the `return .error e` branch in `CapM.run` is provably unreachable. The proof obligation on the caller is the mechanism by which the guarantee is enforced; no `SafeProg` term means no call to `runSafe` means no execution.

3 Parser Report

3.1 The Parser: Architecture and Three-Pass Design

3.1.1 Overview

The job of the parser is to accept raw JSON output from an LLM (which specificies the program) and if the program is legitimate return a `VerifiedProf env`, which is a triple of a result type, `CapM` program and `SafeProg` proof. If there is an error such as an unknown capability name, an authority mismatch or an unbound variable, the parser returns a `ParseError` and the interpreter is not called. There are two challenges here. The first is that we need to generate a proof on the fly while constructing a type of `CapM`. The second is that we need to make sure to support programs where values can flow between commands. For example a read-then-write program takes the string returned from the read as input to the write. Inspired by Cedar's architecture which separates type elaboration and evaluation, we split the parser into three layers Here is an example of how a read-then-write program would look like in JSON and as a `CapM` type.

```
{
  "kind": "let_read",
  "var": "x",
  "capability": "report read",
  "body": {
    "kind": "write",
    "capability": "summary write",
    "contents": {"kind": "var", "name":
    ↪  "x"}
  }
}
```

→

```
CapM.cons (CapCmd.read readCap) (fun s ⇒
  CapM.cons (CapCmd.write writeCap s)
  ↪  CapM.pure)
```

Here `x` is a data flow binding. Hence, we need to track which of these bindings are currently live. When we enter the body of a `let_read` the scope becomes `["x"]` and if the body contains a nested let_read "y", then the scope inside that body would become ["y", "x"].

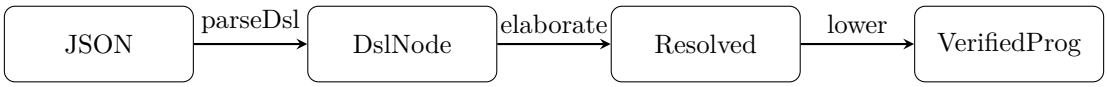

Figure 1: Compiler Pipeline

3.1.2 Pass 1: `parseDsl` — JSON Shape Validation

This converts raw JSON into a `DslNode` which is the unresolved typed AST. It only checks if the DSL obeys the JSON contract. This layer only detects errors of malformed JSON structure, missing fields or unkown kind values.

```
inductive Contents where
  | lit (s : String)
  | var (name : String)

inductive DslNode where
  | read (capName : String)
  | write (capName : String) (contents : Contents)
  | delete (capName : String)
  | seq (first rest : DslNode)
  | letRead (varName capName : String) (body : DslNode)
```

3.1.3 Pass 2: `elaborate` — Resolution and Scoping

`elaborate` converts a `DslNode` into a `Resolved sc`, a typed, scope-indexed AST where capability names have been replaced by actual `Cap` tokens and variable names have been replaced by indices.

The `Scope` and `Resolved` types

```
abbrev Scope := List String   -- variable names currently in scope

inductive RContents (sc : Scope) where
  | lit (s : String)
  | var (idx : Fin sc.length)

inductive Resolved : Scope → Type where
  | read (c : Cap)                                : Resolved sc
  | write (c : Cap) (contents : RContents sc)     : Resolved sc
  | delete  (c : Cap)                               : Resolved sc
  | seq (first rest : Resolved sc)                : Resolved sc
  | letRead (c : Cap) (body : Resolved (name :: sc)) : Resolved sc
```

`Resolved` is indexed by `sc : Scope`. Every node in the tree is parametrised by the set of variable names currently in scope at that point in the program. This is a dependent type: the `letRead`

constructor's `body` field has type `Resolved (name :: sc)` — the body lives in an extended scope where the bound variable is visible. A `Resolved` value is only well-formed if its scope index is consistent throughout.

A `Fin sc.length` is a natural number together with a proof that it is strictly less than the length of the scope. Hence, an out of bounds variable reference cannot be represented as a `Resolved` value.

What `elaborate` does

```
def elaborate (m : ResolveMap) :
    (sc : Scope) → DslNode → Except ParseError (Resolved sc)
```

For each node, `elaborate` performs three checks:

1. **Capability resolution.** It looks up the capability name in the `ResolveMap` — a list of `(String × Cap)` pairs maintained by the orchestrator.
2. **Authority checking.** The resolved `Cap` carries an `authority` field.

1. **Variable resolution.** For a `Contents.var name`, `elaborate` walks the current `sc` to find `name` and converts it to a `Fin sc.length` index. An unresolved name returns `ParseError.unboundVar`.

Scope threading Scope is managed for compound nodes as follows

```
| sc, .seq first rest ⇒ do
    let rFirst ← elaborate m sc first
    let rRest  ← elaborate m sc rest
    return .seq rFirst rRest
| sc, .letRead varName capName body ⇒ do
    let c ← resolveCap m capName
    let _ ← requireAuthority c .read capName
    let rBody ← elaborate m (varName :: sc) body
    return .letRead c rBody
```

Since `seq` runs programs one after another(i.e no nesting here), a variable that is bound in the first part does not enter the second part. This is enforced since both `rFirst` and `rRest` are of type `Resolved sc` for the same `sc`.

`letRead` extends the scope by `varName :: sc` before recursing into the body. The recursive call produces a `Resolved (varName :: sc)`, which is the type the `letRead` constructor expects for its body field. The extension is locally scoped: after the recursion returns, the outer scope is still `sc`.

No proofs are construced here. All errors here are related to variable scope and unknown capabilities.

3.1.4 Pass 3: `lower` — Proof Construction

`lower` constructs the proof that the program is safe under the given environment `CapEnv`.

The return type

```
abbrev ContextValues (sc : Scope) := Fin sc.length → String

structure VerifiedProgOf (env : CapEnv) (α : Type) where
  prog  : CapM α
  hSafe : SafeProg env prog

def lower (env : CapEnv) : {sc : Scope} → Resolved sc →
    Option (Σ α : Type, ContextValues sc → VerifiedProgOf env α)
```

There are two things to unpack here.

`ContextValues sc` is a function from `Fin sc.length` to `String`. It maps each variable index to the runtime string value that variable will hold.

The Σ type. The `Σ α : Type, ...` is an existential: the result type α of the program is not fixed in advance — it depends on the program's structure (`String` for a read-heavy program, `Unit` for a write/delete). The function `ContextValues sc → VerifiedProgOf env` α says: given the runtime values of all variables currently in scope, produce a concrete `CapM` α program together with its `SafeProg` proof.

The key invariant that makes `lower` work `SafeProg.writeSafe env c s hv ha` holds for any string `s`. The predicate constrains capabilities and authorities, never runtime strings. This property makes it possible to produce a `ContextValues sc → VerifiedProgOf env` α at parse time.

```
| writeSafe (env : CapEnv) (c : Cap) (s : String)
    (hv : env.valid c) (ha : c.authority = .write) :
    SafeProg env (CapM.write c s)
```

The `letRead` case

```
| .letRead c body ⇒
    if h : env.valid c then
      if ha : c.authority = .read then
        match lower env body with
        | none ⇒ none
        | some ⟨β, bodyFn⟩ ⇒
            some ⟨β, fun vs ⇒
              ⟨(CapM.read c).flatMap (fun s ⇒ (bodyFn (Fin.cases s vs)).prog),
```

```
              SafeProg.flatMapSafe env (CapM.read c)
                (SafeProg.readSafe env c h ha)
                (fun s ⇒ (bodyFn (Fin.cases s vs)).hSafe)⟩⟩
      else none
    else none
```

The recursive call on `body` returns `bodyFn : ContextValues (name :: sc) → VerifiedProgOf env` β. The body's context has one more variable than the outer scope — the newly bound name.

`Fin.cases s vs` constructs the extended context: given a runtime string `s` (the value the read will return) and the outer `vs : Fin sc.length → String`, it builds a `Fin (sc.length + 1) → String` that maps index `0` to `s` and every other index `i+1` to `vs i`. This is the standard elimination principle for `Fin (n+1)`.

The proof term uses `SafeProg.flatMapSafe`:

```
| flatMapSafe (env : CapEnv) (x : CapM α)
    (h1 : SafeProg env x)
    (h2 : ∀ a, SafeProg env (f a)) :
    SafeProg env (CapM.flatMap f x)
```

3.1.5 Top-Level Entry Point

```
def parseAndVerify (env : CapEnv) (m : ResolveMap) (j : Json) :
    Except ParseError (VerifiedProg env) :=
  match parseDsl j with
  | .error e ⇒ .error e
  | .ok node ⇒
      match elaborate m [] node with
      | .error e ⇒ .error e
      | .ok resolved ⇒
          match lower env resolved with
          | none          ⇒ .error (.invalidCap "<post-elaboration>")
          | some ⟨α, fn⟩ ⇒
              let vp := fn (fun i ⇒ i.elim0)
              .ok ⟨α, vp.prog, vp.hSafe⟩
```

The three passes are sequenced as a pipeline. The result is a concrete `VerifiedProg env` ready for `runSafe`.